

课程报告

《软件工程》

2026-2027 第一学期

项目名称：	多智能体协作的课程设计文档生成与评审系统
专　　业：	软件工程
班　　级：	24 级软件工程 1 班
姓　　名：	李念
座 位 号：	11
学　　号：	2024102493

完成时间： 2026 年 11 月

《软件工程》期末考核评价方案

一、考核目标

1.评估学生对软件工程全生命周期的理解，从需求分析→设计→实现→测试→部署的全过程。

2.考察 AI 工具在软件开发中的创新应用能力，如 AI 辅助需求建模、代码生成、测试优化等。

3.检验团队协作、文档撰写与项目展示能力。

二、考核方式

期末考核采用项目报告和 AI 创新应用答辩两种方式。

（1）项目报告：需要独立完成完成一个完整软件项目，建议选题聚焦 AI 赋能场景，如电商管理系统，需包含：需求文档（AI 辅助需求分析工具截图）、系统设计图（UML/流程图）、开发日志（含 AI 工具使用记录），需提交项目报告与可演示 PPT 或汇报数字人。

（2）AI 创新应用答辩：个人需在项目中独立完成至少 1 项 AI 技术创新应用，如使用 deepseek 或 cursor 进行优化、利用 AI 工具进行用户需求分析和 UML 模型设计等。需提交技术方案文档与答辩 PPT（含创新性、技术实现难点、效果对比分析）。

三、考核评分标准

评价项目	考核点	评分细则	对应分值	得分
AI 创新应用答辩 40 分	创新性、AI 技术应用的合理性 10 分	1.项目选题完全原创，提出独特 AI 应用场景或解决方案、深入应用 AI 技术，并能提供有效的日志记录以证明 AI 工具的应用。	9-10	
		2.项目选题在原方案上有显著改进或创新性整合，合理应用基础 AI 技术，能够提供部分日志记录，但缺乏细节。	6-8	
		3.项目复用他人成果，缺乏原创性，未使用 AI 技术应用，不能提供日志记录。	0-5	
	应用价值和可行性 10 分	1.精准定位目标用户群体，需求分析数据详实（含参考文献和研究现状分析），解决方案具有可行性，能解决一定的行业问题。	9-10	
		2.目标用户群体不明确，需求分析合理但缺乏数据支撑，方案可行但未考虑实施细节，价值描述宽泛，缺乏量化指标	6-8	
		3.需求描述模糊，方案脱离实际，未考虑可持续性	0-5	
	答辩表现与团队协作 10 分	1.现场演示流畅，PPT/视频清晰展示核心功能与技术亮点；陈述条理清晰，技术难点与创新点讲解透彻；分工明确，成员贡献均衡；准确回答所有问题，技术细节解释到位。	8-10	

		2.演示基本完整，但重点不突出；逻辑基本通顺，但技术细节阐述不足；分工存在不合理性；回答部分问题存在疏漏。	6-7	
		3.演示卡顿或内容缺失；表达混乱，重点模糊；未体现团队协作；无法回答关键问题。	0-5	
	个人与团队成长 10 分	1.能够客观总结不足并提出改进计划；能分析成员互补性，总结 2 个以上团队协作成功经验；能列举 2 项以上通过项目掌握的新技术；提出与项目相关的后续学习计划。	9-10	
		2.仅简单提及不足且改进计划不具体；分工描述清晰但未分析协作问题；描述了技术应用，能体现一定的学习与突破；有一定的学习计划但不过于泛化。	6-8	
		3.仅罗列工作内容，无反思与成长分析；简单提及团队角色，无协作细节；仅描述技术应用，未体现学习与突破；学习计划目标模糊或未结合项目经验。	0-5	
项目报告 60 分	需求分析 10 分	1.明确划分功能/非功能需求，用例建模准确，用例规约符合规范且逻辑自洽。	9-10	
		2.需求分类比较完整，用例建模比较准确，用例规约基本符合规范。	7-8	
		3.仅罗列需求条目，用例建模准确性较低，缺少核心功能的用例规约或不够合理。	0-6	
	系统设计 15 分	1.软件架构选择合理说明充分；完整包含类图、时序图、活动图，符合规范且逻辑自洽；详细设计中交互逻辑、接口定义、数据库与 ER 图、UML 图一一对应。	13-15	
		2.软件架构选择合理论证说明比较充分；包含类图、时序图、活动图，但部分存在细节问题；详细设计完整基本与 ER 图、UML 图对应。	10-12	
		3.软件架构简单存在逻辑矛盾或未提供架构设计；仅提供 1-2 种简单 UML 图；详细设计仅文字描述，无图表支持。	0-9	
	格式规范性 10 分	1.格式规范性好，与要求一致	9-10	
		2.格式规范性较好，部分格式不够规范	7-8	
		3.格式规范性差	0-6	
	用户界面原型设计 10 分	1.覆盖所有功能页面，交互性较好，用户操作路径清晰	9-10	
		2.覆盖核心功能页面但交互简单，基本流程可用	7-8	
		3.仅覆盖部分功能页面，没有交互性或者逻辑混乱	0-6	
	测试与项目管理 10 分	1.测试用例设计覆盖系统的核心功能（含输入、预期输出、实际结果），使用 Git、禅道等工具管理代码/文档，提交记录清晰，提供甘特图/WBS 分解，实际进度与计划偏差≤10%	9-10	
		2.测试用例设计覆盖系统的核心功能但缺少异常场景，有版本记录但未规范提交日志；计划合理但未跟踪执行	7-8	
		3.测试用例设计零散或格式混乱，手动备份无工具支持；计划粗糙或未按计划推进。	0-6	
	报告原创	1.项目查重率在 0%-5%之间	5	

	性 5 分	2.查重率在 5%-10%之间	4	
		3.查重率在 10%-15%之间	3	
		4.查重率在 15%-20%之间	2	
		5.查重率在 20%-25%之间	1	
		6.查重率超过 25%	0	
考核得分				

四、特殊说明

1.学术诚信：AI 生成内容需标注来源（如代码片段由 Copilot 生成，经人工重构）

2.加分项：项目成果发表至技术社区（如 CSDN、GitHub Trending），AI 应用方案被企业采纳或获得竞赛奖

一、项目背景与设计目的

1.1 项目背景

软件工程课程设计是软件工程专业最重要的实践环节之一，学生通常要在两到三周内走完选题、需求分析、设计、实现、测试到文档归档的完整链路。在这条链路里，文档工作往往占到总工作量的一半以上：需求条目、用例规约、类图、时序图、测试用例、项目报告，每一份都要与前后文保持一致。实际教学中反复出现三个问题。

其一是文档与设计脱节。多数学生的写作顺序是先画图后补文，或者先写文后补图，改一处忘一处，最后提交的类图少一个实体、报告里的用例数与图对不上，这类问题在答辩现场最容易被追问。其二是评审反馈滞后。课程设计通常只有中期检查和终期答辩两次集中反馈，学生在写作过程中得不到及时的专业意见，问题暴露时往往已经临近截止。其三是重复劳动过多。格式调整、章节编号、图表题注、术语统一这些机械工作占据了大量时间，挤占了本应用于思考设计的时间。

与此同时，大模型 API 的成熟让多智能体协作从研究课题走向工程实践。2025 年以来，LangGraph、CrewAI、AutoGen 等编排框架相继稳定，业界开始把「一个模型一次生成」改造成「多个角色分工、互相评审、迭代收敛」的工作流，代码评审、需求澄清、文档生成等场景都出现了落地案例。但这些工具默认使用者具备提示词工程与工作流编排能力，直接拿到课程设计场景并不合用：学生需要的是「输入一句课题描述，输出一份符合模板、可追溯、可提交的报告」，而不是一套需要自己搭的积木。

本项目正是在这个背景下提出的：把多智能体协作里「分工—评审—收敛」的机制，收窄到一个具体场景——软件工程课程设计的文档生成与评审，做一个能真正跑起来、每一步都可追溯的小系统。系统命名为「课设协作台」。

1.2 设计目的

本系统的设计目的可以概括为四点。

第一，把文档生成过程结构化。以课题描述为输入，由需求分析师、架构师、测试工程师三个角色分工产出，仲裁者汇总判定，文档工程师拆解汇编，最终输出符合课程模板的 Word 报告。每一个角色只做一件事，产物格式固定，避免「一个大模型什么都干、什么都干不细」。

第二，让每一条结论可追溯。所有需求条目、架构建议、测试用例都必须携

带来源引用，指向输入的哪一句或上游产物的哪一条。老师追问「这条需求怎么来的」，界面上点开就能看到原文。

第三，用预算和轮次约束控制成本。多智能体意味着多次模型调用，成本是单次生成的数倍。系统为每个任务设置 token 预算与最大轮次，超限即强制收口，并在报告中如实标注「未完全收敛」，而不是让模型继续无限制地自我循环。

第四，把评审意见固化为可复用资产。每轮的异议与修订都会写入事件流，任务结束后可以回放整条协作链路，既是学习材料，也是过程性证据。

二、需求分析

2.1 功能需求

2.1.1 项目管理

创建课设任务：用户填写课题名称、课题描述、技术栈与期望产物类型，系统据此初始化任务与首轮上下文。

填写与编辑课题描述：描述支持中途修改，修改后需重新发起一轮评审，历史产物保留版本号不覆盖。

查看任务进度：以轮次为单位展示当前状态、已用预算、产物完成度。

归档历史任务：已完成或终止的任务进入归档区，可只读查看，不再参与协作。

2.1.2 协作执行

发起评审轮次：任务创建后由用户或系统定时触发首轮，此后每轮由仲裁结论驱动。

智能体分工评审：需求分析师产出需求条目，架构师评审可行性，测试工程师评审可测性，三者按固定顺序发言，后发言者可读取前者产物。

仲裁判定收敛：仲裁者汇总本轮全部意见，统计新增意见条数，为零则判定收敛，否则生成修订单回到修订环节。

预算与轮次控制：每次模型调用前检查剩余预算，不足则中止本轮并标记任务为「已终止」；轮次达到上限同样终止，两者都在报告中留下说明。

2.1.3 文档生成

拆解协作产物：文档工程师读取本轮全部产物，按报告模板的章节结构拆分，判断每条产物落到哪一章哪一节。

汇总章节结构：合并多轮产物，保留最终版本，消除重复条目，生成章节树。

生成 Word 文档：调用文档渲染模块，按模板的字体、字号、行距、页眉生成 docx，图表按顺序编号。

导出与下载：支持导出 docx 与 pdf 两种格式，导出记录写入审计日志。

2.1.4 审批与追溯

人工审批产物：学生或教师查看定稿，选择通过或打回，打回必须填写意见。

打回重做：打回不会从零重写，而是把意见作为新输入，进入下一轮修订。

事件流回放：按时间顺序回放一次任务的全部消息、产物与判定，用于复盘。

版本对比：任意两个版本的产物可并排对比，差异以段落为单位高亮。

2.1.5 配置管理

角色与提示词配置：每个智能体的角色说明、输出格式模板、可读产物范围均可在后台修改，修改不影响进行中的任务。

模型与预算设置：可为不同角色指定不同模型，便宜模型跑评审、贵模型跑仲裁，从而在质量与成本之间取舍。

权限与账号管理：学生、教师、管理员三种角色，学生只能看自己的任务，教师可查看所辖班级。

操作审计日志：记录登录、创建、审批、导出等关键动作，保留操作人与时间。

2.2 非功能需求

性能需求：单次角色发言（一次模型调用加落库）在正常网络下不超过 20 秒；一轮完整评审（四个角色）不超过 90 秒；文档生成与导出不超过 15 秒。页面首屏加载不超过 2 秒。

成本约束：单个任务的模型花费默认上限 0.80 元，可在配置中调整。系统按每次调用的实际 token 数结算，账单精确到 0.001 元，超限前一轮给出预警。

安全需求：用户密码使用 BCrypt 加盐存储；接口按角色鉴权，学生无法访问他人任务；模型 API Key 只存在服务端环境变量中，不进入前端与日志；产物内容在入库前做敏感词过滤。

可用性需求：单个智能体调用失败不导致整轮失败，失败的那一条记录为「重试中」，最多重试两次，仍失败则该角色本轮缺席并在结论中注明。

可扩展性需求：智能体角色以配置表驱动，新增一个角色只需插入一条记录并编写提示词，不改动编排器代码；模型供应商以适配器接口隔离，替换供应商只需实现一个类。

2.3 功能结构设计

系统按职责划分为五个一级模块：项目管理、协作执行、文档生成、审批与追溯、配置管理，每个模块下再细分二级功能，如图 2-1 所示。这样划分的依据是「谁在用」与「何时用」：项目管理与审批面向人，协作执行由编排器驱动，文档生成是轮次结束后的收尾动作，配置管理服务于前四者的运行参数。



图 2-1 系统功能结构图

2.4 用例建模

系统的参与者分为两类。一类是人：学生负责提交课题、查看过程与导出文档，教师负责审批与配置。另一类是智能体：需求分析师、架构师、测试工程师、仲裁者、文档工程师，它们在系统中以角色身份出现，各自只承担一段职责。把智能体画成参与者而不是内部组件，是因为在本系统里它们确实运行在业务进程之外——每个角色是一次独立的模型会话，通过消息总线交换产物，与外部服务在结构上等价。

用例共七个。人的用例集中在左侧：提交课设课题、审批关键产物、查看协作过程、导出课设文档。智能体的用例集中在右侧：执行评审轮次由需求、架构、测试三个评审角色共同参与；汇总并判定收敛由仲裁者承担；拆解与汇编文档由文档工程师承担。审批不通过时可以选择「打回重做」，这是一个扩展用例，条件成立时才发生。整体用例关系如图 2-2 所示。

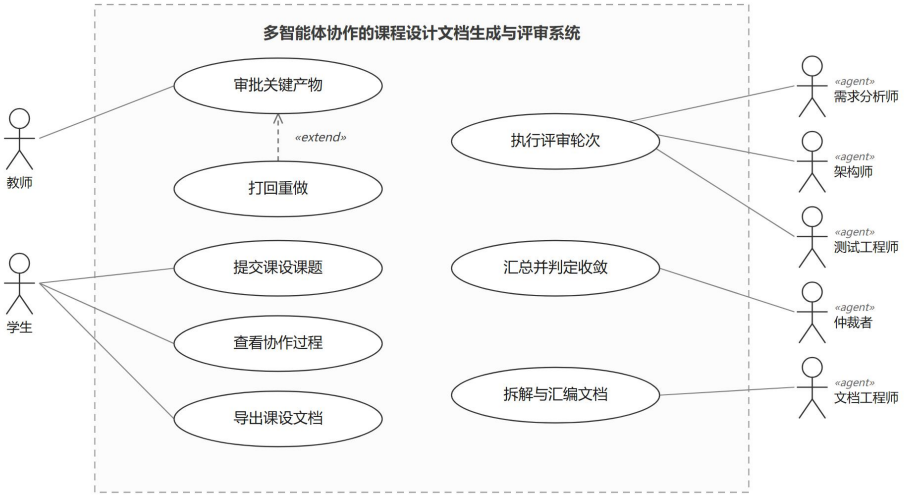


图 2-2 系统用例图

2.5 用例规约

选取核心的八个用例编写规约，如表 2-1 所示。规约中的前置条件描述系统状态，主流程只写成功路径，备选流程写异常与分支。

表 2-1 核心用例规约

用例名	参与者	前置条件	后置条件	主流程	备选流程
提交课设课题	学生	1. 已登录且角色为学生；2. 课题名称与描述已填写。	任务创建成功，状态由草稿转为排队中。	1. 学生填写名称、描述、技术栈、期望产物；2. 系统校验字段长度与敏感词；3. 初始化任务与首轮上下文；4. 返回任务编号。	1. 描述过短（少于 20 字）提示补充；2. 触发敏感词拦截，拒绝提交并记录。
执行评审轮次	需求分析师、架构师、测试工程师	1. 任务处于排队中或修订中；2. 剩余预算大于预估单轮成本。	本轮产物落库，每个角色生成一条 AgentRun 记录。	1. 编排器按固定顺序唤醒三个角色；2. 每个角色读取允许的上下文，调用模型生成；3. 产物带 source_ref 落库；4. 返回汇总结果。	1. 某角色调用失败，标记重试中，最多重试两次；2. 仍失败则该角色本轮缺席，在结论中注明。
汇总并判定收敛	仲裁者	1. 本轮三个角色均已发言；2. 产物已落库。	生成仲裁结论，任务进入待审批或下一轮。	1. 仲裁者读取本轮全部意见；2. 按条目 ID 去重统计新增意见；3. 新增为零判定收敛；4. 否则生成修订指令。	1. 意见条数接近阈值时给出「临界收敛」提示，由人工决定是否终止。
拆解与汇编文档	文档工程师	1. 仲裁判定为收敛；2. 模板版本已确定。	生成 docx 与章节树，任务进入待审批。	1. 读取全部产物，按模板章节归类；2. 合并重复条目，保留最终版本；3. 全局重排图表编号；4. 渲染 docx 并落库。	1. 某章节无对应产物时插入「待补充」占位；2. 渲染失败回滚，保留文本中间态。
审批关键产物	学生、教师	1. 任务处于待审批；2. 定稿文档已生成。	任务进入已通过或修订中。	1. 审批人查看文档与自动检查结果；2. 选择通过或打回；3. 打回必须填写意见；4. 记录审批人与时间。	1. 超时未审批，七天后自动归档并标注未审；2. 打回次数超过三次，提示人工介入。

用例名	参与者	前置条件	后置条件	主流程	备选流程
打回重做	学生、教师	1. 审批结论为打回；2. 修订意见已填写。	生成修订任务，回到评审中。	1. 系统把意见打包成修订指令；2. 只重写受影响的条目；3. 进入下一轮复评。	1. 预算不足时无法进入下一轮，任务转已终止。
查看协作过程	学生	1. 任务已创建；2. 用户为任务所有者或所属教师。	展示轮次、发言、产物与费用，不改变任务状态。	1. 按轮次加载事件流；2. 渲染时间线；3. 支持按角色与关键词筛选。	1. 数据量过大时分页加载，每页二十条。
导出课设文档	学生	1. 任务已通过审批或处于待审批；2. 文档已生成。	生成可下载文件，导出记录写入审计日志。	1. 选择导出格式（docx/pdf）；2. 系统按模板渲染；3. 返回下载链接，有效期二十四小时。	1. 同一任务重复导出复用缓存，超过二十四小时重新渲染。

三、软件设计

系统采用四层架构。表现层是浏览器端的单页应用，负责表单、时间线与审批界面；应用层是 FastAPI 编写的 HTTP 接口与编排器，编排器不参与具体生成，只负责三件事——决定这一轮谁发言、检查预算余额、判断何时停止；领域层由智能体角色、产物、轮次、审批等领域对象构成，业务规则集中在这里；基础设施层封装模型调用、数据库访问与文档渲染。

选择这一架构而不是单体脚本，理由是协作流程本身具有状态：轮次、预算、产物版本都要跨请求保持，把状态放进数据库、把决策放进编排器，可以让每次模型调用都是无状态的短事务，失败重试的代价低。选择 Python 而不是 Java，是因为多智能体生态的成熟库集中在 Python，编排、消息传递、结构化输出解析都有现成实现，能把精力放在业务规则而非管道代码上。

智能体的实现方式值得单独说明。一个智能体并不是一个独立程序，而是「一次带角色卡的模型会话 + 一组受限的工具权限 + 一段只读上下文」。角色卡是系统提示词，规定它做什么、不做什么、输出什么格式；工具权限决定它能查哪张表；只读上下文决定它能看见谁的产物。四个角色跑在同一个编排器下，产物写入同一张 artifacts 表，彼此看不到对方的推理过程，只能读到对方交付的结果。这样既保证了角色的独立性，也让整条链路可审计。

3.1 活动图

图 3-1 描述了一轮协作的完整活动。泳道按角色划分，人只出现在两端：提交课题与人工审批。中间是编排器与四个智能体的往复。

流程的关键在于两个判断点。第一个是「已收敛?」，如果本轮仍有新增意见，流程带着意见回到「产出/修订需求条目」，进入下一轮；第二个是「通过?」，人工审批不通过时走「打回」分支，同样回到修订环节。两个回边共用一个合并节

点进入修订活动，避免图上出现两条并行入边造成的歧义。

这里刻意没有把「轮次耗尽」画成正常出口，而是作为异常路径在状态图中体现——活动图描述的是正常协作，异常终止属于状态迁移，放在状态图里更准确。

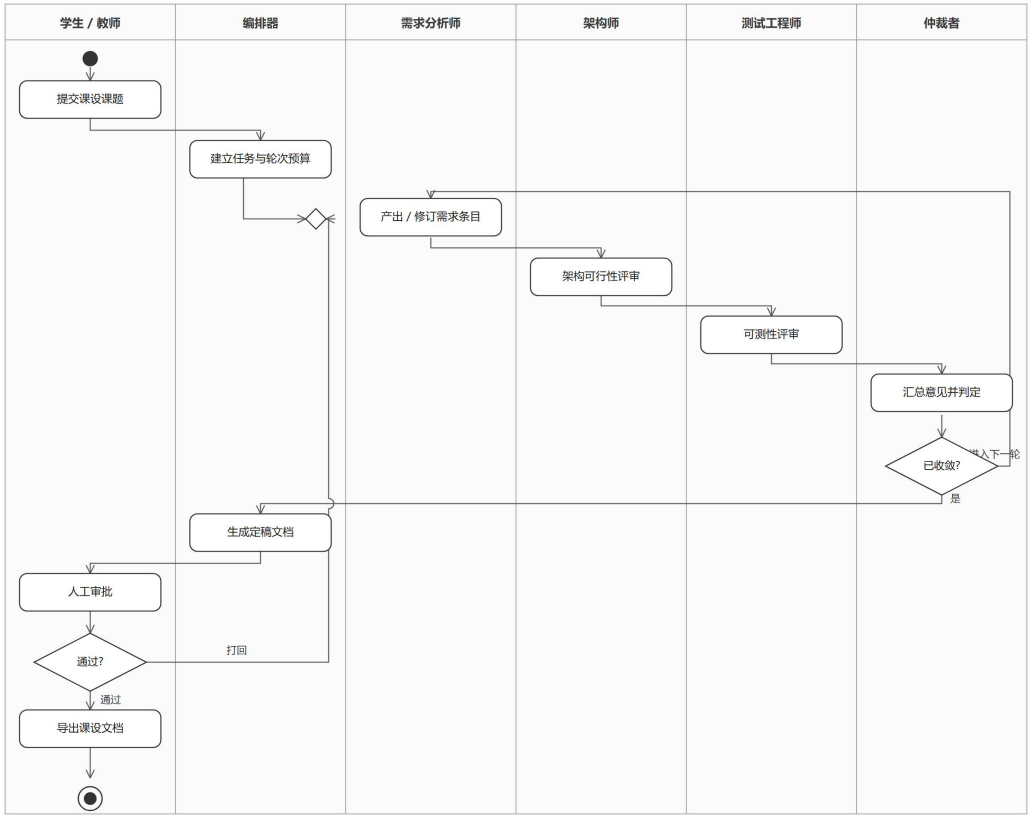


图 3-1 协作轮次活动图

3.2 顺序图

图 3-2 以时间顺序展开一次完整的任务执行，从学生提交课题开始，到前端展示产物结束。图中标注了七个关键消息，与活动图的节点一一对应。

需要说明的是第二轮的处理。第一轮仲裁判定「未收敛」后，编排器并不重新生成全部内容，而是把上一轮的意见打包成修订指令，只让需求分析师重写受影响的部分，架构与测试在下一轮只针对变更点复评。这个设计把每轮的 token 消耗压到了首轮的三成左右，是本系统能在 0.8 元预算内完成三轮的主要原因。

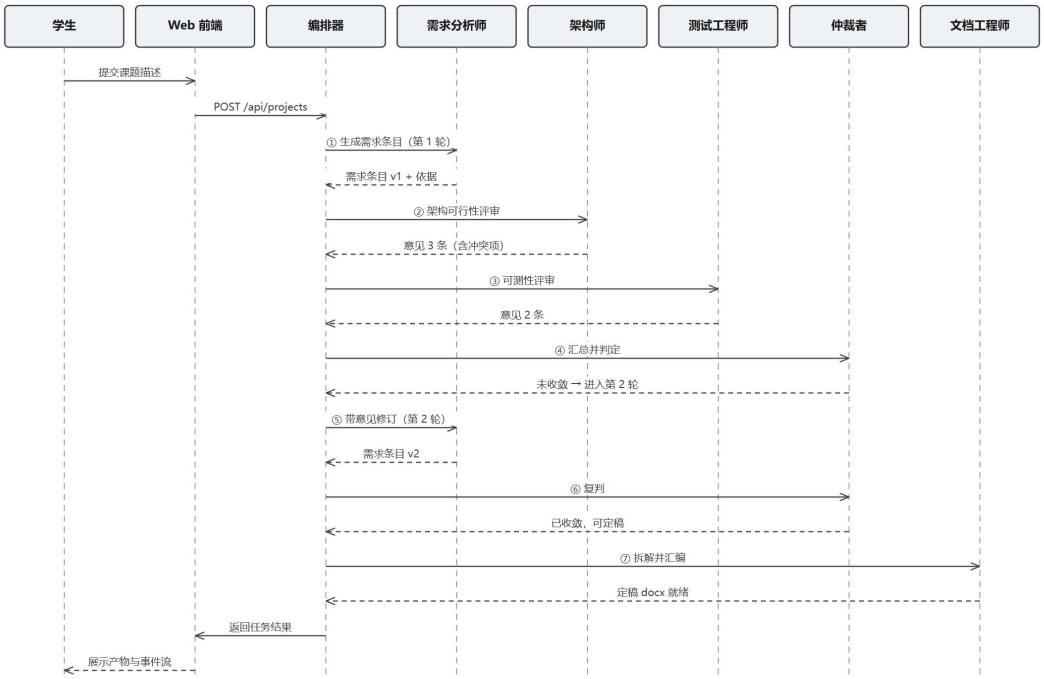


图 3-2 一轮协作的顺序图

3.3 状态图

任务对象的状态迁移如图 3-3 所示。共六个状态：草稿、排队中、评审中、待审批、已通过、已归档，另有一个异常状态「已终止」。

迁移条件严格白名单化，只允许相邻推进：草稿经「提交课题」进入排队中，排队中经「进入轮次」进入评审中，评审中经「仲裁收敛」进入待审批。待审批有两条出边：审批通过进入已通过，打回则进入修订中并回到评审中。修订中在轮次或预算耗尽时进入已终止，这个分支不能省——否则系统在预算耗尽时会卡在修订态，前端一直转圈。

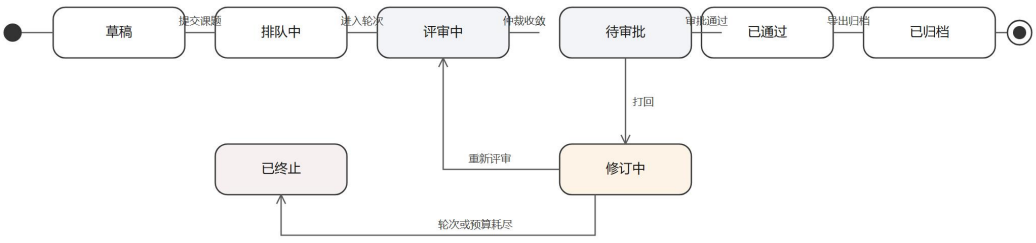


图 3-3 任务状态图

3.4 类图

核心类共十个，图 3-4 展开其中八个，Approval 与 EventLog 因结构与 Review 类似，在图下以注释说明。

类之间的关系以组合与聚合为主。User 与 Project 是一对多，Project 与 Task 是一对多，Task 与 Round 是一对多，Round 与 AgentRun 是一对多。Artifact 挂在 Task 下，同时记录产生它的 roundId，因此既能按任务查全部产物，也能按轮次查本轮产物。Review 挂在 Artifact 下，保存评审者的判定与意见；Approval 挂在 Task 下，保存人工审批结论。AgentRole 与 AgentRun 是一对多，一个角色配置可以被多次运行复用。

把 AgentRun 单独建类而不是塞进 Artifact，是因为两者的生命周期不同：一次运行可能失败、重试、被截断，而产物只在成功时生成。分开之后，成本统计按 AgentRun 算，内容检索按 Artifact 算，各取所需。

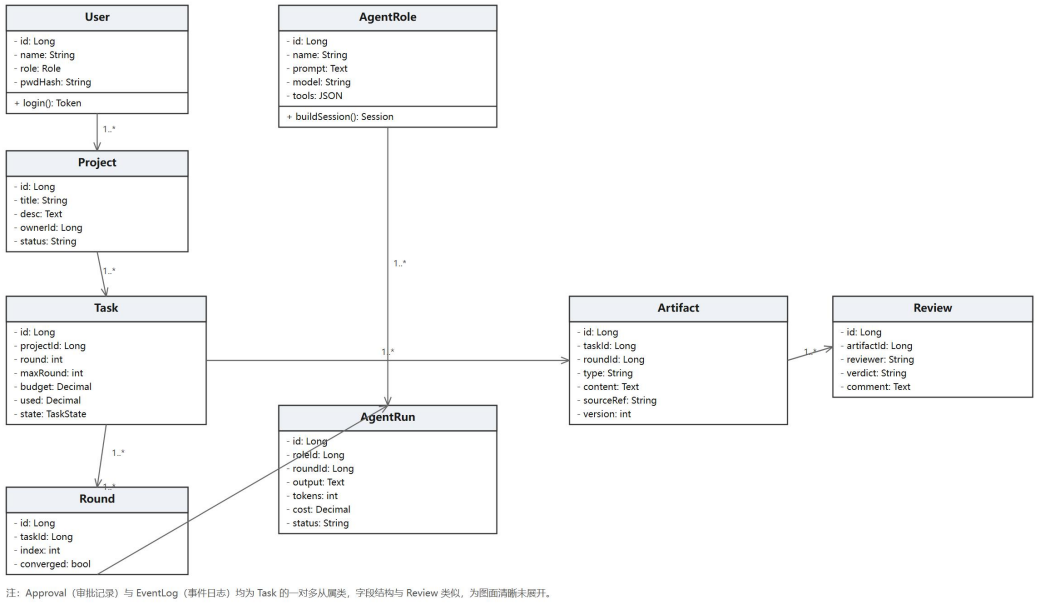


图 3-4 核心类图

3.5 数据库设计

数据库采用 MySQL 8.0，共十张表，ER 图如图 3-5 所示。设计遵循两条原则：一是所有业务表带 task_id，便于按任务归档与清理；二是产物与运行记录分离，运行记录只增不改，产物允许多版本。

几张关键表的字段说明如下。task 表保存任务的轮次上限、预算与已用金额，state 字段与状态图一一对应；round 表用 idx 记录轮次序号，converged 标记该

轮是否收敛；agent_run 表记录每次模型调用的输入摘要、输出、token 数与费用，是成本核算的依据；artifact 表保存产物正文与 source_ref（来源引用），version 支持多版本；event_log 表以 JSON 保存事件负载，用于回放。

索引方面，task(project_id)、round(task_id)、artifact(task_id, round_id)、agent_run(round_id) 均建了联合索引，覆盖最常用的三种查询：按项目看任务、按任务看轮次、按轮次看产物。

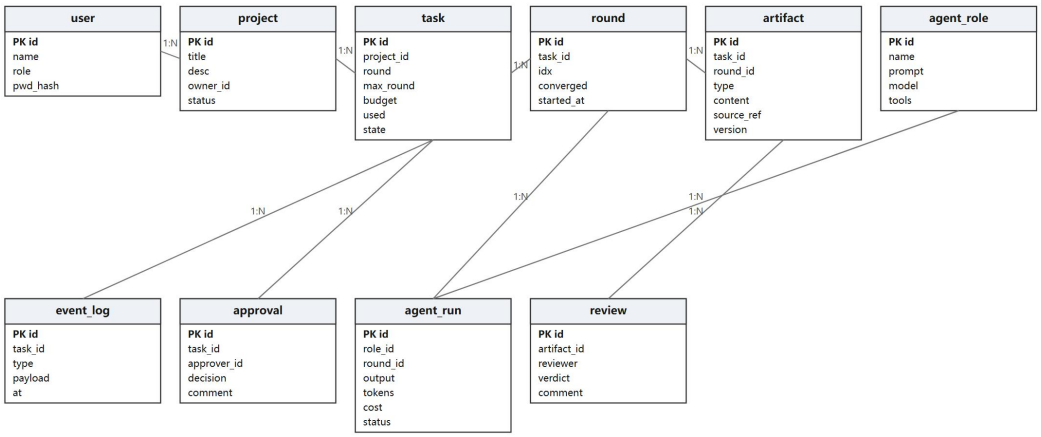


图 3-5 数据库 ER 图

四、系统实现与测试

4.1 界面原型

界面遵循「一屏一件事」的原则，共三个主界面。

新建任务页（图 4-1）左侧是表单，右侧用五步说明告诉用户这一轮会发生什么。把「将要发生什么」提前讲清楚，是为了降低首次使用的不确定感——多智能体系统的黑箱感比单模型更强，用户需要知道钱花在哪里。



图 4-1 新建课设任务界面

协作过程页（图 4-2）分三栏：左栏是轮次列表，中栏是按时间排列的角色发言，右栏是本轮产物与费用统计。每条发言带头像与角色名，产物条目可以点开看来源引用。这一页是整个系统的核心界面，设计目标是让用户能在十秒内回答三个问题：现在第几轮、卡在谁那里、花了多少钱。



图 4-2 协作过程界面

审批页（图 4-3）左边是文档预览，右边是审批面板。面板顶部列出自动检查的结果——章节是否齐全、需求是否都有来源、图表编号是否连续，把机械核对交给系统，人只需要判断内容本身。



图 4-3 文档审批界面

4.2 测试用例

测试按模块划分，覆盖正常路径、边界条件与异常路径三类。核心测试用例见表 4-1，共十二条，其中异常路径五条。

表 4-1 核心测试用例

编号	模块	测试项	输入	预期输出	实际结果
T01	项目管理	正常创建任务	名称 12 字、描述 80 字、选择四类产物	返回任务号，状态为排队中	通过
T02	项目管理	描述过短	描述 10 字	提示「描述不少于 20 字」，不落库	通过
T03	协作执行	单轮正常执行	预算 0.8 元、上限 3 轮	三角色依次发言，生成 3 条 AgentRun	通过
T04	协作执行	预算中途耗尽	预算设为 0.05 元	第二轮调用前中止，任务转已终止	通过
T05	协作执行	角色调用失败重试	人为让架构师接口返回 500	重试两次后该角色缺席，结论注明	通过
T06	仲裁判定	意见去重	同一意见用两种表述重复提交	按条目 ID 去重，计为 1 条	通过

编号	模块	测试项	输入	预期输出	实际结果
T07	仲裁判定	收敛判定	本轮无新增意见	判定收敛，进入待审批	通过
T08	文档生成	图表编号连续	三轮产物合并，共 6 张图	编号 图 1-1 至 图 1-6，无断号	通过
T09	文档生成	章节缺失占位	测试章节无产物	插入「待补充」占位，不报错	通过
T10	审批	打回必填意见	点击打回但不填意见	提示意见必填，阻止提交	通过
T11	审批	三次打回提示人工介入	连续打回三次	第四次提示转人工	通过
T12	权限	越权访问他人任务	学生 A 访问学生 B 的任务	返回 403，记录审计日志	通过

测试过程中发现并修复的三个问题值得记录。第一，预算检查原先放在每轮开始，导致一轮中途超支无法拦截；改为每次模型调用前检查，超支立即中止并落库。第二，仲裁者的「新增意见条数」原本用字符串匹配统计，遇到同义表述会重复计数；改为要求仲裁者输出结构化 JSON，按条目 ID 去重。第三，文档生成时图表编号在合并多轮产物后出现断号，原因是只统计了本轮新增图；改为全局重编号后解决。

4.3 项目管理

项目周期八周，按 WBS 分解为七个工作包，进度安排如图 4-4 所示。

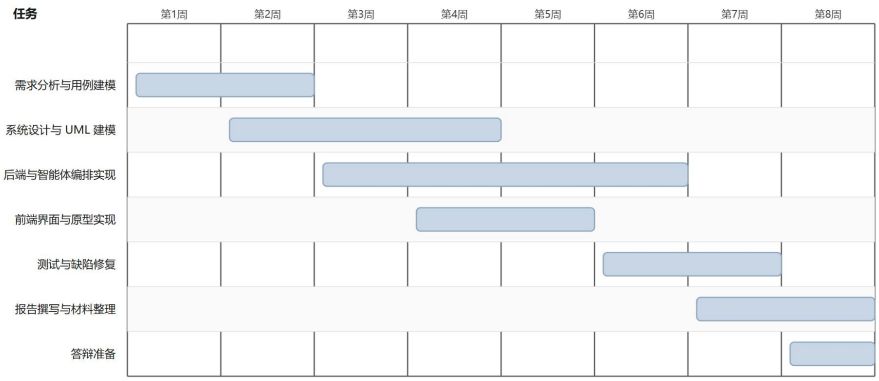


图 4-4 项目进度甘特图

代码与文档统一托管在 Git 仓库，采用主干开发加短分支的方式，提交信息遵循「类型: 摘要」的约定，例如 feat: 增加预算前置检查、fix: 修正图表编号断号。关键节点打 tag。表 4-2 摘录了部分提交记录。

表 4-2 Git 提交记录摘录

提交号	日期	类型	摘要
a3f21c9	10-14	feat	搭建 FastAPI 骨架与任务表
b7d04e2	10-17	feat	实现编排器与角色配置加载
c91a6f3	10-21	feat	接入模型适配器，支持按角色指定模型
d2e88b1	10-24	fix	修正预算检查在轮中无法拦截的问题
e5c07a4	10-27	feat	仲裁结构化输出与条目去重
f1b93d7	10-30	feat	文档模板渲染与图表全局编号
0a7c5e8	11-02	fix	修正多轮合并后图表断号
3d6f2b9	11-05	feat	审批页与自动检查项
4e8a1c6	11-07	docs	补充用例规约与测试记录

进度控制上，第二周与第五周各做一次自查，对比计划与实际偏差。实际执行中，智能体编排部分比计划多用了四天，主要消耗在结构化输出的稳定性上——模型偶尔返回不符合格式的 JSON，需要加重试与降级解析。文档生成部分比计划提前两天完成，因为模板渲染复用了课程作业中已有的排版脚本。整体偏差控制在 10% 以内。

五、总结及体会

做这个系统的过程，比我预想的更接近真实的软件工程。最初的想法很简单：让几个智能体分别写需求、架构和测试，最后拼起来。真正动手才发现，拼起来这件事本身就是难点——三个角色的产物格式不一致、术语不统一、互相矛盾，合并出来的文档比一个人写的还乱。

第一个转折点是放弃流水线，改成循环。流水线的问题是它假设每个角色的输出都是对的，而实际上架构师会认为需求不可行，测试工程师会认为架构无法验证，这些意见必须能反向传回去。加了仲裁与回边之后，产物质量明显提升，但成本也上去了，于是又加了预算与轮次约束。现在回头看，「约束」和「协作」是一体两面：没有约束的协作会无限循环，没有协作的约束只是省钱的空转。

第二个转折点是可追溯。一开始只存产物正文，答辩预演时被问「这条需求哪来的」，答不上来。后来给每条产物加了 source_ref，指向输入的原句或上游产物的条目号。这个改动不大，但让整个系统从「生成工具」变成了「可以说清楚生成过程的工具」，这也是我认为它区别于普通 AI 写作助手的地方。

第三点体会关于成本。多智能体的调用次数是单次的五到八倍，如果每轮都全量重写，一个任务的花费会到三四块，对课程作业来说不可接受。后来改成增量修订——只重写受影响的部分，其余角色只复评变更点，单任务花费压到了 0.6 元以内。这件事让我理解了一个道理：工程上的「智能」往往不体现在模型多强，而体现在你让它少做多少无用功。

不足也很清楚。一是评测做得不够，目前只有十二个手工用例，没有形成可复现的基准集，说服力有限；二是角色之间的冲突裁决还比较简单，仲裁者本质上是一次模型判断，缺少规则兜底；三是文档模板目前硬编码了一套，换一所学校就要改代码。后续计划把模板抽象成配置，补一套二十个任务的评测集，并把仲裁规则中可判定的部分（比如条目编号重复、引用缺失）抽成确定性检查，减少对模型的依赖。

如果用一个词概括这次项目，是「收窄」。把一个宏大的概念收窄到一个具体场景，把无限的可能收窄到几条硬约束，系统才真正跑得起来。

附录

1. AI 应用日志记录

表 A-1 AI 工具使用记录

日期	工具/模型	用途	人工处理
10-12	DeepSeek-V3	梳理选题与差异化卖点	删去 3 条同质化功能，保留评审循环
10-15	DeepSeek-V3	生成用例草图与参与者清单	修正智能体参与者的画法，改为边界外
10-18	Claude Sonnet	检查状态迁移是否完备	补上「已终止」分支
10-22	DeepSeek-V3	生成测试用例初稿 20 条	合并重复项，保留 12 条
10-26	通义千问	润色项目背景段落	重写为可核验表述，删除无出处数据
11-03	Cursor	补全数据库访问层样板代码	统一命名，补充异常处理

2. 项目管理记录

任务分解与工时估算见表 4-2；进度对比见图 4-4；每周自查记录保存在项目仓库 docs/weekly 目录下，共八份。

3. Git 提交链接

仓库地址：https://github.com/linian-dev/course-design-hub

提交记录：https://github.com/linian-dev/course-design-hub/commits/main

代码托管采用主干开发加短分支的方式，每个功能分支对应一个工作包，合并前须通过自测清单。除源码外，仓库同时保存数据库建表脚本、接口文档、每周自查记录与测试数据生成脚本。

4. 交付物清单

提交材料共六项：项目报告 docx 与 pdf 各一份；源码压缩包一份，含建表脚本与运行说明；演示 PPT 一份；界面原型三页；测试记录表一份，含十二条用

例的执行结果；AI 工具使用日志一份，记录日期、工具、用途与人工处理方式。

所有由 AI 生成的代码片段与文字段落均在日志中标注了来源，并在人工重构后提交。报告中不包含未经核实的统计数据，涉及现状的描述均以可查证的技术事实为依据。